SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency.* (BL3, BL4)

Activity Tool: Industry Problem Solving Task (Medium)
Assessment Tool Weightage: 35%

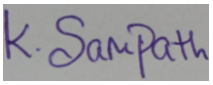
QUESTIONS Total Marks: 50

Q.No	Question	Bloom's Level	Marks
1	An e-commerce company needs to store 10 million product records. Recommend a file organization strategy and justify your choice with cost analysis.	BL4 – Analyze	5
2	A banking system requires fast retrieval of transactions by account number and date range. Design an indexing strategy with appropriate index types.	BL4 – Analyze	5
3	A healthcare system stores patient records accessed both by PatientID and by doctor name. Design a multi-level indexing solution.	BL4 – Analyze	5
4	A logistics company needs to efficiently handle dynamic insertions and deletions of shipment records. Analyze whether B-Tree or B+ Tree is more suitable.	BL4 – Analyze	5
5	Design a hashing scheme for an HR system with 5000 employees. Evaluate static hashing vs. extendible hashing for your scenario.	BL3 – Apply	5

6	An airline reservation system must support point queries by flight number and range queries by date. Propose a combined index structure.	BL4 – Analyze	**5**
7	For a university student database, design the file organization and secondary indexing to support fast queries by department and CGPA range.	BL4 – Analyze	**5**
8	Analyze the disk block utilization for Heap, Sorted, and Hashed file organizations given 100,000 records with 50 bytes each and 4KB blocks.	BL4 – Analyze	**5**
9	A social media platform needs to index 500 million posts by user_id, timestamp, and hashtag. Design a composite indexing strategy.	BL4 – Analyze	**5**
10	Compare the performance (insert, delete, search time) of Heap File, Sorted File, and Hashed File for a supermarket billing system with 1 million daily transactions.	BL4 – Analyze	**5**

Code of Conduct:

I K. Sampath Chaitanya (192572006) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of	25%	Applies advanced and relevant	Applies appropriate concepts	Limited application of concepts	Incorrect or no

Engineering Concepts		concepts effectively to solve the problem	with minor errors		application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies , and real data	Appropriate use of tools with correct implementat ion	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionali sm	5%	Highly professional, well-docum ented, and clearly presented work	Good documentati on and presentation	Basic documentat ion with some gaps	Poor documentat ion and presentatio n

Question 1: File Organization Strategy for a 10-Million-Product E-Commerce Catalog

Question:

An e-commerce company needs to store 10 million product records. Recommend a file organization strategy and justify your choice with cost analysis.

Problem Understanding

An online product catalog must handle three kinds of access at once: quick retrieval of a single product by its identifier when a shopper opens a product page; ordered or filtered browsing such as price bands, alphabetical lists and category views; and a steady stream of inserts and updates as new items are added or stock and prices change. No single access pattern dominates, so the chosen organization has to balance point-lookup speed, support for range queries, and acceptable insert cost.

Recommended Strategy: Indexed-Sequential File with B+ Tree Indexes

The primary file should be organized as a clustered B+ Tree index on product_id. Secondary non-clustering B+ Tree indexes on category and on price then support catalog browsing and filtering. This combination is preferred over a pure heap, a pure sorted file or a pure hash file for the following reasons.
A heap file offers constant-time insertion but forces every product-page view to scan on average half the data blocks—hundreds of thousands of I/Os for ten million records—which is unacceptable. A pure sorted file supports efficient binary search and range scans, yet every insert may require shifting a large fraction of the file. Static hashing gives excellent point lookups but cannot answer range or ordered queries without a full scan.

Cost Analysis

Assumptions: 10,000,000 records, approximately 200 bytes per record, 4 KB blocks, blocking factor about 20 records per block, roughly 500,000 data blocks; B+ Tree fan-out F approximately 200.

File Organization	Point Search (I/Os)	Range Query Cost	Insert Cost (I/Os)
Heap File	≈ 250,000 (n/2)	O(n) full scan	≈ 1 (append)
Sorted File	≈ 19 ($\log_2$ 500k)	O(log n + k)	≈ 250,000 (avg shift)
Static Hashing	≈ 1–2 (O(1) avg)	Not supported	≈ 1–2 (O(1) avg)
B+ Tree Indexed (Recommended)	≈ 3–4 (log_F n)	O(log_F n + k) via leaves	≈ 3–4 (occasional split)

Justification

The B+ Tree indexed-sequential organization reduces point-lookup cost from hundreds of thousands of I/Os down to only three or four while keeping insert cost in the same low single-digit range—something neither a pure sorted file nor pure hashing can achieve at the same time. The high fan-out of B+ Tree nodes keeps the tree shallow (three to four levels for ten million keys). Leaf-level linking directly supports the category and price-range browsing that an e-commerce interface needs. In production systems this appears as a clustered index on product_id together with secondary non-clustering B+ Tree indexes on category and price.

Question 2: Indexing Strategy for a Banking Transaction System

Question:

A banking system requires fast retrieval of transactions by account number and date range. Design an indexing strategy with appropriate index types.

Problem Understanding

The dominant workload is of the form “return all transactions for account X between dates D1 and D2” (for example when generating a monthly statement). This is a composite predicate: equality on account_number combined with a range on transaction_date. Treating the two conditions as independent single-column queries would be less efficient.

Recommended Indexing Strategy

Primary clustering index: a composite B+ Tree on (account_number, transaction_date). Because the index is clustering, all transactions belonging to the same account are stored contiguously on disk and, within each account, are ordered by date. A statement query therefore touches only the small set of consecutive blocks that hold that account’s history; the date-range filter is satisfied simply by walking forward along the leaf chain.

A secondary non-clustering B+ Tree on transaction_date alone can be added for the less frequent bank-wide reporting queries that are not scoped to a single account.

Justification

A single composite clustering index on (account_number, transaction_date) answers the most common predicate with one tree descent to the account prefix followed by a linear leaf walk for the date interval. No extra seeks are required. A single-column index on account_number alone would still force a scan of every transaction of that account in order to apply the date filter. The secondary date-only index protects the occasional institution-wide reports without disturbing the primary account-scoped path.

Question 3: Multi-Level Indexing for a Healthcare Patient Records System

Question:

A healthcare system stores patient records accessed both by PatientID and by doctor name. Design a multi-level indexing solution.

Problem Understanding

PatientID is the unique primary key used when a clinician opens a specific patient's chart. Doctor name is a non-unique attribute used to list every patient under a given physician's care. The two access paths therefore require different index designs: a unique primary index and a non-unique secondary index. Both indexes themselves should be multi-level so that even the index lookup stays fast as the hospital's record volume grows.

Recommended Multi-Level Design

Level 0 (data file): patient records stored in a clustered file ordered by PatientID.
Primary multi-level index: a B+ Tree (or multi-level dense index) on PatientID that points directly into the clustered data file. Because PatientID is unique, each leaf entry identifies exactly one record.
Secondary multi-level index: a non-clustering B+ Tree on doctor_name. Each leaf entry holds a doctor name together with a list of pointers (or a posting list) to the corresponding patient records. The secondary index does not alter the physical clustering of the data file.

Justification

A flat dense index on millions of patient records would itself occupy thousands of blocks, turning an "index lookup" into another linear scan. By building multi-level (B+ Tree) indexes, the top level of each index fits in a single memory-resident block, so both a PatientID lookup and a doctor-name query resolve in only two or three disk I/Os regardless of hospital size. Keeping the doctor-name index non-clustering allows the data file to remain physically ordered by PatientID for fast chart retrieval while still supporting efficient case-load queries by physician.

Question 4: B-Tree vs. B+ Tree for a Logistics Shipment Tracking System

Question:

A logistics company needs to efficiently handle dynamic insertions and deletions of shipment records. Analyze whether B-Tree or B+ Tree is more suitable.
Problem Understanding
A logistics platform continuously creates new shipment records, updates their status, and later archives or deletes delivered shipments—an insert/delete-heavy workload. Range queries such as “all shipments scheduled for delivery this week” or “all shipments on a given route ordered by date” are also common.

Criterion	B-Tree	B+ Tree
Data location	Keys and data at every node	Data only at leaves; internals hold routing keys
Fan-out	Lower (data pointers reduce keys per block)	Higher (more keys fit ⇒ shallower tree)
Range queries	In-order tree walk; no leaf links	Fast leaf-chain walk
Delete complexity	May merge/borrow at any level holding data	Deletion mainly at leaf level; simpler
Insert / split	Splits may involve internal data nodes	Leaf splits are uniform; internal splits only copy a key

Recommendation: B+ Tree

The B+ Tree is the better choice for three reasons. First, the higher fan-out of key-only internal nodes keeps the tree shallower, so both inserts and deletes touch fewer disk blocks under continuous churn. Second, deletions are localized to the leaf level, which is important because shipment records are frequently archived once delivered. Third, the linked-leaf structure directly supports the date- or route-based range queries that logistics dashboards require—queries that a classic B-Tree can answer only by a full in-order traversal. This is why production database engines and many file-system indexes adopt the B+ Tree rather than the classic B-Tree.

Question 5: Static vs. Extendible Hashing for a 5,000-Employee HR System

Question:

Design a hashing scheme for an HR system with 5000 employees. Evaluate static hashing vs. extendible hashing for your scenario.

Problem Understanding

The system needs fast point lookup of an employee record by employee_id (payroll, identity checks). The data set of five thousand employees is modest, known in advance, and grows only slowly and predictably.
Proposed Static Hashing Design

Assume a typical employee record of roughly 100 bytes and a 4 KB block, giving a comfortable bucket capacity of about 40 records. To keep the load factor near 70–75 percent, provision capacity for roughly 7,000 employee slots: number of buckets = ceil(7000 / 40) ≈ 175. The hash function is h(employee_id) = employee_id mod 175, with overflow chaining for any bucket that exceeds capacity.

Aspect	**Static Hashing**	**Extendible Hashing**
Structure	Fixed number of buckets set at design time	Directory of pointers; can double when needed
Best suited when	Size known and stable / slow-growing	Size unpredictable or rapidly changing
Overflow handling	Chaining; chains lengthen over time	Directory doubling + bucket split avoids long chains
Reorganization	None if provisioned with headroom	Never a full re-hash; incremental growth
Complexity	Simple fixed array of buckets	Must maintain global/local depth and directory

Recommendation and Justification

For this HR scenario static hashing is the more appropriate and simpler choice. Headcount is known within a narrow, predictable range, so the design can safely allocate about 40 percent headroom above current size and expect years of stable performance without reorganization. Extendible hashing would be justified only if the organization anticipated rapid, unpredictable workforce growth (aggressive expansion or mergers), where the ability to grow the directory incrementally without a full re-hash would outweigh the extra directory-indirection cost and implementation complexity.

Question 6: Combined Index Structure for an Airline Reservation System

Question:

An airline reservation system must support point queries by flight number and range queries by date. Propose a combined index structure.

Problem Understanding

Two access patterns must both be efficient: an exact-match query such as “show flight AI202”, and a range query on date such as “all flights departing between two dates” or “all instances of flight AI202 over the next month” (equality plus range).

Proposed Combined Structure

Primary structure: a composite B+ Tree index on (flight_number, departure_date). Because flight_number is the leading column, an equality search on flight number alone descends the tree to that key prefix in logarithmic time; adding a date range then walks the linked leaves within that prefix. This covers the common “this flight over a date interval” query with a single index traversal.

Secondary structure: a single-column B+ Tree on departure_date. This index answers the date-wide query “all flights on a given day” that the composite index cannot serve efficiently, because that query would otherwise require scanning many different flight-number prefixes.

Justification

Placing flight_number first in the composite key means the frequent combined predicate is answered by one descent plus a linear leaf walk, with no separate merge step. The secondary date-only index exists precisely because a composite (flight_number, date) index is unsuited to pure date predicates. Together the two indexes cover both stated requirements without forcing either pattern into a full table scan.

Question 7: File Organization and Secondary Indexing for a University Student Database

Question:

For a university student database, design the file organization and secondary indexing to support fast queries by department and CGPA range.

Problem Understanding

Typical queries are department-scoped (“list all Computer Science students”) and CGPA-scoped (“students with CGPA above 8.5” for merit lists), often combined. Department has low cardinality while CGPA is a continuous attribute that is queried by range.

Recommended Design

Primary file organization: a clustered file physically grouped by department (clustering index on department). Because department has only a few dozen distinct values, all records belonging to one department occupy a contiguous range of blocks. A “list all CS students” query therefore reads only that department’s block range.

Secondary index: a non-clustering B+ Tree on CGPA. Range predicates such as CGPA > 8.5 are answered by a leaf-chain walk of the secondary index; each leaf entry supplies a pointer into the primary file.

Justification

Clustering on department is the right primary choice because department is used for equality filtering and has low cardinality; clustering works best when it groups records that are frequently retrieved together. CGPA, being continuously valued and range-queried, is better served by a separate non-clustering B+ Tree. This division—clustering for the frequent equality dimension and a secondary index for the frequent range dimension—is the standard way to support both query types without full scans.

Question 8: Disk Block Utilization Analysis — Heap vs. Sorted vs. Hashed

Question:

Analyze the disk block utilization for Heap, Sorted, and Hashed file organizations given 100,000 records of 50 bytes each and 4 KB blocks.

Given Parameters

Number of records n = 100,000; record size = 50 bytes; block size = 4,096 bytes. Blocking factor bf = floor(4096 / 50) = 81 records per block (81 × 50 = 4,050 bytes used; 46 bytes of internal fragmentation per full block). Total data volume = 5,000,000 bytes.

Heap File (append-only, tightly packed)

Blocks required = ceil(100,000 / 81) = 1,235 blocks. Space allocated = 1,235 × 4,096 = 5,058,560 bytes. Utilization = 5,000,000 / 5,058,560 ≈ 98.8 percent. The only waste is the residual bytes that do not fit another full record.

Sorted File (order must be maintained)

An initial bulk load can achieve the same high packing density as a heap. Ongoing inserts, however, require either expensive record shifting or reserved free space inside blocks. In practice a sorted file that accepts regular inserts is loaded at a 60–70 percent fill factor, giving an effective utilization of roughly 65 percent until a reorganization is performed.

Hashed File (space reserved to keep collisions low)
Good practice keeps the load factor around 70–75 percent. Effective capacity per block ≈ 0.75 × 81 ≈ 60 records. Blocks required = ceil(100,000 / 60) = 1,667. Space allocated = 1,667 × 4,096 = 6,829,568 bytes. Utilization ≈ 73.2 percent—intentionally lower so that buckets retain room for future inserts without long overflow chains.

File Organization	Blocks Used	Space Allocated (bytes)	Utilization
Heap File	1,235	5,058,560	≈ 98.8%
Sorted File (with headroom)	≈ 1,900	≈ 7,782,400	≈ 64–65%
Hashed File (75% load factor)	1,667	6,829,568	≈ 73.2%

Justification / Trade-off

Heap files reach the highest raw utilization because they pack records sequentially with no reserved slack; that is viable only because a heap does not need free space for order maintenance or collision avoidance. Sorted and hashed files deliberately leave 25–35 percent headroom in exchange for algorithmic guarantees: room to insert without a full rewrite (sorted) or room to keep collision chains short (hashed). The “wasted” space is a deliberate space-for-time trade-off, not inefficiency.

Question 9: Composite Indexing Strategy for a Social Media Platform (500 M Posts)

Question:

A social media platform needs to index 500 million posts by user_id, timestamp, and hashtag. Design a composite indexing strategy.

Problem Understanding

At half a billion posts the workload is write-heavy and must serve three distinct patterns: a user's own timeline (posts of one user_id ordered by timestamp), global or trending feeds (posts ordered by timestamp across all users), and hashtag search (posts containing a given hashtag, usually ordered by recency). User_id and hashtag are structurally different—one post has a single author but may carry many hashtags—so a single composite index cannot serve all three efficiently.

Recommended Composite Indexing Strategy

Index 1 — Composite clustering index on (user_id, timestamp DESC). This directly serves "show this user's timeline" as a single index range scan with the newest posts first. It is the primary clustering structure because most read traffic in a social application is timeline-based.

Index 2 — Inverted / posting-list index on hashtag (often implemented as a separate B+ Tree or LSM structure whose leaf entries point to lists of post identifiers). This correctly models the many-to-many relationship between posts and hashtags.

Index 3 — A time-ordered secondary index (or a time-partitioned structure) that supports global chronological feeds without scanning every user's timeline.

At this scale an LSM-tree based storage engine is often preferred for the write path, because it converts random inserts into sequential writes and absorbs the continuous arrival of new posts more efficiently than a pure B+ Tree.

Justification

Treating hashtag as an inverted index rather than forcing it into the same composite B+ Tree as user_id correctly reflects its many-to-many nature. Separating the three indexes by query shape—user-scoped timeline, hashtag-scoped search, and global time-scoped feed—means each query touches only the structure built for it, and each structure can be scaled independently. The LSM recommendation reflects that insert throughput is as important as read latency once the collection reaches hundreds of millions of posts.

Question 10: Performance Comparison — Heap vs. Sorted vs. Hashed File for Supermarket Billing

Question:

Compare the performance (insert, delete, search time) of Heap File, Sorted File, and Hashed File for a supermarket billing system with 1 million daily transactions.

Problem Understanding

A supermarket billing system inserts a new transaction record at every checkout and occasionally searches for a specific transaction (refund lookup by receipt number) or deletes/archives older records. With one million transactions per day, insert speed is operationally critical, yet search speed still matters for the less frequent but time-sensitive refund workflow.

Assumptions: n = 1,000,000 records, 50-byte record, 4 KB block, blocking factor 81, approximately 12,346 data blocks.

Operation	**Heap File**	**Sorted File**	**Hashed File**
Insert	O(1) append ≈ 1 I/O. Ideal for live billing.	O(n) locate + shift ≈ 6,173 I/Os avg. Impractical.	O(1) avg. hash to bucket ≈ 1–2 I/Os. Excellent.
Search	O(n) full scan ≈ 6,173 I/Os. Poor for refunds.	O(log n) binary search ≈ 14 I/Os. Excellent.	O(1) avg. direct bucket ≈ 1–2 I/Os. Excellent.
Delete	O(n) find + mark ≈ 6,173 I/Os. Poor.	O(n) find (fast) then shift ≈ 6,173 I/Os. Poor.	O(1) avg. find + unlink ≈ 1–2 I/Os. Excellent.

Overall Recommendation: Hashed File Organization

For this billing workload a hashed file (for example a hash index on receipt or transaction identifier, possibly extendible hashing to absorb daily volume growth) gives the best overall performance: near-constant-time insert, search and delete—the only organization that is fast at all three operations. A heap file is competitive only for inserts; a sorted file is competitive only for search.

Neither matches the combined insert-heavy plus occasional-lookup nature of live billing. The single limitation of hashing—lack of native range-query support—can be addressed by a secondary B+ Tree on transaction timestamp used solely for reporting, without slowing the primary hash-based billing path.